

Deep Learning for Sequential Graph Construction: A Behavior Cloning Study on Minimum Spanning Trees

Davi de Aragão¹

Abstract—Sequential graph construction provides a useful perspective for studying how learning-based policies make structured connectivity decisions over time. This work investigates whether Deep Learning can imitate deterministic edge-selection trajectories in a controlled Minimum Spanning Tree (MST) setting. We formulate MST construction as a supervised sequential decision problem and train a Graph Neural Network (GNN) policy through behavior cloning. Each graph state is represented by a three-channel tensor encoding the current partial tree, graph connectivity, and normalized edge distances. Expert MST solutions are converted into deterministic edge-selection sequences and expanded into state-action transitions for supervised learning. The resulting agent combines dense-to-sparse graph preprocessing, edge-aware message passing, edge-level action decoding, and an explicit END prediction. Experiments on 100,000 synthetic connected weighted graphs show that the best highlighted checkpoint achieves 72.37% validation accuracy and 72.47% test accuracy on next-action prediction, compared with approximately 3.5% for a random valid-action baseline and 32% for a simple greedy acyclic baseline. In a supplementary rollout analysis over 100 graphs, the learned policy produced valid spanning trees in all cases, matched the expert MST cost in 52 cases, and achieved an average relative cost gap of 0.56%. These results show that a GNN-based behavior-cloning pipeline can learn non-trivial sequential graph-construction behavior in a controlled MST benchmark, while also establishing a reproducible baseline for future work on stronger evaluation protocols, broader graph families, and reinforcement-learning refinement.

I. INTRODUCTION

Optimization is one of the fundamental pillars of modern engineering. Designing communication networks, transportation systems, energy infrastructures, manufacturing processes, robotic missions, and logistics operations all require identifying configurations that optimize one or more performance goals while satisfying complex physical and operational constraints. In many of these domains, the underlying decision problem can be naturally represented as a graph, where vertices model system entities and edges encode feasible interactions, communication links, spatial relationships, or transportation routes [?], [?], [?]. Because key performance indicators such as throughput, latency, routing efficiency, structural quality, and deployment cost depend strongly on how nodes are interconnected, graph topology optimization has become a fundamental computational problem in communication network design, urban mobility, infrastructure planning, autonomous systems, and scientific computing [?], [?], [?], [?].

Graph topology optimization is difficult for two related reasons. First, the search space is inherently combinatorial and may grow rapidly with the number of admissible links or structural decisions [?], [?]. Second, practical formulations often involve nontrivial constraints, nonlinear interactions, or multiple competing goals, which make large-scale optimization even harder [?], [?]. For this reason, graph topology optimization problems have been approached through several complementary paradigms, including exact algorithms [?], [?], heuristic and metaheuristic methods [?], [?], mathematical programming [?], [?], and, more recently, learning-based approaches [?], [?]. Exact methods provide optimality guarantees whenever tractable formulations are available [?], [?], whereas heuristic and metaheuristic algorithms often offer greater flexibility at the expense of theoretical guarantees [?], [?]. Mathematical programming provides a rigorous optimization framework but typically depends on explicit formulations and may become computationally prohibitive for large or highly structured instances [?], [?]. These limitations have motivated increasing interest in data-driven approaches capable of learning reusable optimization strategies from previously solved problems rather than solving every new instance from scratch [?], [?], [?].

Recent advances in Deep Learning have significantly accelerated the adoption of learning-based approaches for graph optimization. Instead of explicitly encoding optimization rules, neural models can learn decision policies directly from data, enabling fast inference once training is completed [?], [?], [?]. In contrast to traditional pipelines that repeatedly solve each instance from scratch, learning-based methods can amortize computation across many previously solved examples and then reuse that experience at deployment time [?], [?], [?]. This paradigm has shown promising results in several combinatorial optimization problems, including routing, design, and networked decision making, where learned policies can generalize across problem instances while reducing computational cost during deployment [?], [?], [?], [?].

Among neural architectures, Graph Neural Networks (GNNs) are particularly attractive for graph optimization because they naturally operate over relational structures [?], [?], [?]. Through message passing, GNNs iteratively aggregate information from neighboring vertices and edge attributes, allowing node representations to capture both local interactions and global structural context [?]. This relational inductive bias has made GNNs one of the predominant model families for graph-based learning [?], [?], motivating applications in network design, topology optimization, molecular

¹Aeronautics Institute of Technology (ITA), São José dos Campos, Brazil. davi.aragao@ga.ita.br

reasoning, and neural combinatorial optimization [?], [?], [?], [?], [?].

Existing learning-based approaches to graph topology optimization can be broadly grouped into three paradigms. The first learns graph-to-graph transformations, where an existing graph is iteratively modified to improve structural properties [?], [?], [?]. The second focuses on autoregressive graph generation, in which entirely new graph structures are synthesized through sequential node and edge generation [?]. The third formulates optimization as a sequential decision-making problem over a fixed graph [?], [?], [?], [?], as commonly found in routing and neural combinatorial optimization, where policies predict ordered sequences of decisions to construct paths, tours, or topological improvements [?], [?], [?], [?].

Despite their differences, these paradigms share a common characteristic: graph construction is typically performed either by editing existing topologies, generating entirely new graphs, or producing ordered sequences of graph construction decisions [?], [?], [?], [?], [?]. Comparatively little attention has been devoted to learning edge-by-edge graph construction policies over fully observed graphs through supervised imitation [?]. Such a formulation naturally bridges supervised and reinforcement learning [?], [?]. Expert demonstrations can first be used to learn an initial graph construction policy, which can subsequently be refined through reinforcement learning to optimize objectives for which expert solutions are unavailable [?], [?], [?], [?].

This observation leads to the following research question: can a graph neural network learn to imitate deterministic graph-construction trajectories directly from weighted graph states under a controlled MST benchmark [?]?

Accordingly, we investigate sequential graph construction through a controlled supervised-learning benchmark rather than through a claim of solving general topology optimization. The central contribution of this work is methodological: a complete behavior-cloning pipeline for edge-by-edge MST construction. Additional contributions concern how this pipeline is documented and evaluated as a baseline for later work. Concretely, the contributions are:

- the formulation of graph construction as a supervised sequential decision problem;
- a graph representation that jointly encodes graph connectivity, construction state, and edge distances;
- a graph neural policy architecture for edge-by-edge graph construction;
- an empirical benchmark, including highlighted-checkpoint and rollout analysis, establishing a supervised baseline for future extensions such as stronger evaluation protocols, broader graph families, and later reinforcement-learning refinement.

The remainder of this paper is organized as follows. Section II describes the methodology, including graph generation, graph encoding, transition expansion, policy architecture, and training protocol. Section III reports the experimental results and discusses their implications. Section IV concludes the paper and outlines future work.

II. METHOD

The proposed method follows a pipeline composed of three main stages: dataset generation, data preparation, and policy learning. First, a synthetic dataset of connected weighted graphs is generated together with expert Minimum Spanning Tree (MST) solutions obtained from a classical optimization algorithm. Next, these graph-solution pairs are transformed into supervised state-action transitions by encoding graph states and expanding each expert solution into a sequence of partial construction steps. Finally, a Graph Neural Network (GNN) policy is trained through behavior cloning to predict the next graph-construction action from each observed state. The following subsections describe each stage of the proposed pipeline in detail.

A. Dataset Generation

The first stage of the proposed pipeline consists of generating the data required to train the policy. Each sample is composed of a connected weighted graph together with an expert construction sequence obtained from a classical Minimum Spanning Tree (MST) algorithm. The graph defines the optimization instance, whereas the expert trajectory provides the reference actions used during learning.

a) Synthetic Graph Generation: The graph generator samples nodes uniformly in a two-dimensional square domain. Given a target graph size and an expected degree, the side length of the domain is adapted so that random geometric graphs with a prescribed connection radius are likely to meet the desired edge density. Two vertices are connected when their Euclidean distance is below the connection radius, and the edge weight is exactly that Euclidean distance.

The generator accepts graphs with 8 to 20 nodes and targets between $N - 1$ and 50 edges, with a tolerance of two edges around the desired count. Graphs are rejected and resampled whenever they are disconnected or whenever their realized edge count differs from the target by more than two edges. This process yields a synthetic yet structurally coherent family of weighted graphs in which geometry directly determines the optimization target.

b) Expert Solution Generation: Once a synthetic weighted graph has been generated, an expert construction sequence is produced to serve as the supervision signal for behavior cloning. The expert solution is obtained by applying a classical Minimum Spanning Tree (MST) algorithm implemented through the `networkx.minimum_spanning_tree` routine from NetworkX [?], which computes an optimal spanning tree using the edge weights associated with the graph.

The resulting MST defines the subset of optimal edges but does not specify the order in which these edges should be selected during graph construction. To obtain a deterministic expert trajectory, a depth-first traversal is performed over the spanning tree, visiting neighboring vertices in ascending order of their identifiers. The traversal produces an ordered sequence of edge-selection actions that uniquely defines the expert policy associated with each graph instance.

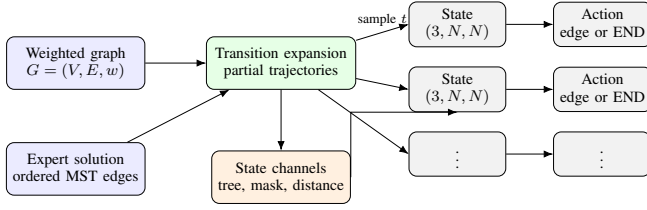


Fig. 1. Transformation of one graph-level expert solution into supervised transitions. Each weighted graph and its expert MST sequence are expanded into multiple partial state-action pairs. Every state is encoded as a three-channel tensor describing the current tree, graph connectivity mask, and normalized edge distances, while the target action corresponds to the next expert edge or the END decision.

Finally, each graph-solution pair is serialized into a compact textual representation of the form

$$u, v, w; u, v, w; \dots \# a_0, a_1, \dots, a_K, \quad (1)$$

where each tuple (u, v, w) represents one weighted edge and the sequence following the separator $\#$ stores the ordered expert actions. This representation is lightweight, human-readable, and contains all information required to reconstruct both the optimization instance and its corresponding expert trajectory.

B. Data Preparation

The goal of the data preparation stage is to transform each expert graph-solution pair into a supervised dataset suitable for policy learning. Conceptually, this requires defining what the agent observes, what decision it must produce at each step, and how a full expert trajectory is decomposed into multiple independent supervision samples. Under this formulation, graph construction becomes a supervised sequential decision problem in which each training example is composed of one graph state and one expert action.

a) State Representation: At decision step t , the agent observes the graph state

$$s_t = (T_t, C, D), \quad (2)$$

where T_t denotes the current partial tree, C is the binary connectivity matrix of the original graph, and D is the normalized edge-distance matrix. The first component changes along the construction trajectory, whereas C and D remain fixed for a given graph instance.

For implementation purposes, this state is encoded as a tensor

$$s_t \in \mathbb{R}^{3 \times N \times N}, \quad (3)$$

with the following channels:

- channel 0: current partial tree;
- channel 1: graph connectivity mask;
- channel 2: normalized edge distances.

Distances are normalized within each graph by the largest edge weight present in that sample. This preserves relative geometry while keeping the numerical scale consistent across episodes.

b) Action Representation: Given a state s_t , the agent must either select one admissible edge or emit a termination decision. Formally, the action at step t satisfies

$$a_t \in \mathcal{E}_t \cup \{\text{END}\}, \quad (4)$$

where $\mathcal{E}_t$ denotes the set of graph edges admissible in the current episode representation. For indexing purposes, if the current graph contains M_i edges, the corresponding local action space is

$$\mathcal{A}_i = \{0, 1, \dots, M_i - 1, \text{END}\}, \quad (5)$$

where actions 0 to $M_i - 1$ index the graph edges of episode i and END terminates the construction process.

Because mini-batch training requires a fixed output size, local actions are mapped to a global padded indexing scheme during training. In the current implementation, states are padded to graphs with at most 20 nodes, so the policy always outputs logits for all

$$\binom{20}{2} + 1 = 191 \quad (6)$$

possible padded actions, corresponding to 190 undirected edge slots plus END. Only the locally valid actions are considered during training and inference through an external action mask.

The behavior cloning objective is the cross-entropy loss over the expert action:

$$\mathcal{L}_{BC}(\theta) = - \sum_{(s_t, a_t^*)} \log \pi_\theta(a_t^* | s_t). \quad (7)$$

c) State-Action Expansion: A complete graph-solution pair is not used directly as one training sample. Instead, it is expanded into a sequence of partial transitions. At step t , the state contains the partial tree built from the first t expert edges, and the target action is the next edge in the expert sequence. After the last edge, an additional END action is added so that the policy also learns when to stop.

Because batching requires a fixed state shape, each sample is padded to the maximum graph size of 20 nodes before being stored in the supervised dataset. Zero padding is used outside the active $N \times N$ region, so padded entries are distinguishable from valid graph entries through the connectivity mask. Across the generated dataset, the largest observed graph contains 52 edges, which implies a maximum observed local action space of 53 when END is included. The original graph-solution pairs are serialized as text strings, whereas the expanded supervised dataset is stored as a PyTorch `.pt` tensor bundle with states represented in `float16` for compactness.

Algorithm 1 summarizes the transition-expansion procedure used to convert each expert trajectory into independent training samples.

C. Policy Learning

The policy learning stage consists of learning a graph construction policy from the supervised state-action dataset generated in the previous stage (Section II-B). The model

Algorithm 1 Transition Expansion from Full Expert Solution

Require: Weighted graph G , ordered expert solution $\tau = (a_0, \dots, a_{K-1})$

Ensure: Transition set $\mathcal{D} = \{(s_t, a_t^*)\}$

- 1: Initialize empty partial tree T_0
- 2: $\mathcal{D} \leftarrow \emptyset$
- 3: **for** $t = 0$ to $K - 1$ **do**
- 4: Build state s_t from T_t , graph connectivity, and normalized distances
- 5: Add transition (s_t, a_t) to $\mathcal{D}$
- 6: Apply action a_t to obtain T_{t+1}
- 7: **end for**
- 8: Build final state s_K from T_K
- 9: Add termination transition (s_K, END) to $\mathcal{D}$
- 10: **return** $\mathcal{D}$

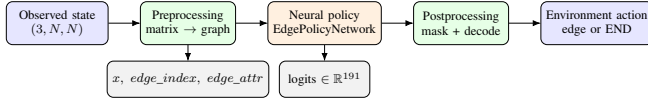


Fig. 2. Graph construction agent. The preprocessing stage encodes the dense matrix state into sparse graph primitives, the neural policy evaluates the graph and outputs padded logits, and the postprocessing stage applies the action mask and decodes the selected action.

is trained to imitate the expert construction process by predicting the next action from the current graph state. The learned policy is implemented as a Graph Neural Network (GNN), whose architecture is described below.

a) Graph Construction Agent: The learning agent should be understood as a decision-making wrapper around the neural policy, rather than as the neural network alone. Given an observed state tensor, the agent first converts the dense matrix representation into the sparse graph format required by the GNN backend. The neural policy then produces one logit for each padded edge slot and one additional termination logit. Finally, an external postprocessing step applies the action mask, removes invalid padded or non-existent edges, and reconstructs the executable environment action. Therefore, the complete agent is composed of three stages: state preprocessing, neural policy evaluation, and action postprocessing.

The preprocessing and postprocessing stages can also be interpreted as an encoder-decoder wrapper around the neural policy. The preprocessing stage encodes the dense matrix observation into sparse graph primitives, whereas the postprocessing stage decodes padded network logits into one valid edge-selection action or the END decision. This distinction is important because the GNN does not directly consume the original matrix state, nor does it directly emit an executable environment action.

b) Policy Architecture: The policy is an edge-selection graph neural network organized into four conceptual blocks: input representation, graph encoding, message passing, and action decoding. The central idea is that the environment is observed as a dense matrix tensor, but the policy reasons over a sparse relational structure. Therefore, the model explicitly

converts the matrix state into graph primitives before applying graph message passing. The architecture is implemented in PyTorch [?] and PyTorch Geometric [?].

In the input-representation block, the state tensor of shape $(3, N, N)$ is decomposed into the current tree matrix, the graph connectivity mask, and the normalized distance matrix. Candidate edges are extracted from the connectivity matrix and converted into the sparse graph representation required by PyTorch Geometric [?]. For each valid undirected edge (u, v) , the raw edge feature vector is

$$\mathbf{e}_{uv} = [\text{tree}_{uv}, \text{dist}_{uv}], \quad (8)$$

where $\text{tree}_{uv} \in \{0, 1\}$ indicates whether the edge already belongs to the partial tree and $\text{dist}_{uv} \in [0, 1]$ is the normalized weight. This encoder transforms the dense matrix state into the sparse tuple $(x, \text{edge_index}, \text{edge_attr})$. For message passing, each undirected edge is duplicated in both directions.

In the input-projection block, all nodes start from a constant scalar feature, so the node projection (`node_encoder` in the implementation) maps $\mathbb{R}^1 \rightarrow \mathbb{R}^H$. The raw edge features are projected independently by an edge projection (`edge_encoder`) that maps $\mathbb{R}^2 \rightarrow \mathbb{R}^H$. The message-passing block is implemented by two consecutive GINEConv layers:

$$\mathbf{h}^{(1)} = \text{GINE}_1(\mathbf{h}^{(0)}, \mathbf{e}), \quad \mathbf{h}^{(2)} = \text{GINE}_2(\mathbf{h}^{(1)}, \mathbf{e}), \quad (9)$$

with LeakyReLU activations after each layer. GINE was selected because it naturally incorporates edge attributes into message passing [?], which is essential in the present setting because edge weights directly determine the optimization target. This stage produces context-aware node embeddings with shape (N, H) that depend jointly on graph structure, current construction state, and edge distances.

In the action-decoding block, node embeddings are converted back into edge-level representations. For each candidate edge (u, v) , the decoder forms

$$\mathbf{z}_{uv} = [\mathbf{h}_u^{(2)}, \mathbf{h}_v^{(2)}, \mathbf{e}_{uv}], \quad (10)$$

which concatenates the latent representations of both incident nodes with the raw edge attributes. This explicit edge embedding has dimension $2H + 2$ and is central to the model because the final action is edge selection, not node classification. The edge embeddings are then refined by a residual multilayer perceptron:

$$\tilde{\mathbf{z}}_{uv} = \mathbf{z}_{uv} + \text{MLP}_{\text{res}}(\mathbf{z}_{uv}), \quad (11)$$

and mapped to one scalar score per candidate edge by an edge head $\mathbb{R}^{2H+2} \rightarrow \mathbb{R}$.

Finally, the policy predicts the END action through a dedicated graph-level termination head. Node embeddings are mean-pooled into a graph embedding of dimension H , which is processed by a small multilayer perceptron $\mathbb{R}^H \rightarrow \mathbb{R}^H \rightarrow \mathbb{R}$ to produce the END logit. The final action vector concatenates all edge logits with the END logit, producing logits in $\mathbb{R}^{191}$. During training and evaluation, invalid actions

are filtered outside the network forward pass through action masking so that optimization and prediction are restricted to graph-valid edges and the END action. The postprocessing stage then decodes the selected global index back into one valid environment action.

Table I summarizes the architecture at the block level used in the implementation. The goal of this summary is not to replace the mathematical description above, but to make the tensor flow and the role of each module explicit in a format that is easier to audit against the code.

c) Training Protocol: The supervised dataset contains 100,000 original graph episodes and 1,283,100 expanded transitions. Splits are performed by episode rather than by transition, preventing leakage between partial states derived from the same graph. The resulting partitions contain 80,000 training graphs, 10,000 validation graphs, and 10,000 test graphs.

All highlighted models were trained with Adam and cross-entropy loss. For compactness, the discussion in this report emphasizes two checkpoints that differ only in hidden dimension, namely 64 and 128. Both were trained for 10 epochs with learning rate 5×10^{-4} , batch size 512, no learning-rate scheduler, no weight decay, deterministic split seed 42, training-data shuffling enabled, and `num_workers=0`. Model checkpoints are selected according to the lowest validation loss, and test-set evaluation is performed only after reloading the corresponding best checkpoint for each configuration. Although the supervised states are stored in `float16` format for compactness, they are cast to the model parameter dtype during optimization, so the training procedure itself does not rely on mixed-precision updates.

The codebase also includes a grid-search utility over learning rate, batch size, and hidden dimension. Specifically, the explored learning rates are $\{10^{-3}, 5 \times 10^{-4}, 2 \times 10^{-4}\}$, the explored batch sizes are $\{512, 1024\}$, and the explored hidden dimensions are $\{64, 128\}$, yielding 12 candidate configurations. Each configuration is trained for 10 epochs and ranked by validation loss. The checkpoints discussed in this report correspond to the highlighted runs in Table IV.

All experiments were implemented in Python 3.11.4 using PyTorch 2.13.0 [?], PyTorch Geometric 2.8.0 [?], NetworkX 3.6.1 [?], and NumPy 2.4.6 [?]. Training and evaluation were executed on an Apple MacBook Pro equipped with an Apple M1 Pro processor and 16 GB of unified memory, using the PyTorch MPS backend for accelerated tensor computation.

d) Implementation Summary: For reproducibility, the main architectural and experimental settings are summarized in Table II. The node encoder maps $\mathbb{R}^1 \rightarrow \mathbb{R}^H$, the edge encoder maps $\mathbb{R}^2 \rightarrow \mathbb{R}^H$, each GINEConv layer maps $\mathbb{R}^H \rightarrow \mathbb{R}^H$ under edge-aware message passing, the residual edge decoder maps $\mathbb{R}^{2H+2} \rightarrow \mathbb{R}^{2H+2}$, the edge-scoring head maps $\mathbb{R}^{2H+2} \rightarrow \mathbb{R}$, and the termination head maps $\mathbb{R}^H \rightarrow \mathbb{R}$. In the retained experiments, $H \in \{64, 128\}$.

III. RESULTS AND DISCUSSION

A. Dataset Characteristics

Table III summarizes the main characteristics of the supervised dataset generated from the original weighted graph collection.

The expansion procedure is central to the learning setup. Rather than observing only complete expert trees, the model is trained on partial construction states. This turns one graph-level optimization target into many supervised decisions and makes the constructive process directly learnable.

B. Next-Action Prediction Performance

Table IV compares the two highlighted checkpoints and two simple reference baselines. The uniform random baseline corresponds to the expected accuracy of sampling one locally valid action uniformly among the admissible graph edges and END. Its value is not a fixed constant of the model, but a consequence of the average number of valid actions across the evaluation transitions; this expectation was computed exactly over the full validation and test splits. The greedy acyclic baseline predicts the lightest currently admissible edge, or END if the tree is already complete. Because this baseline is implemented procedurally rather than batched, it was estimated over the first 5,000 transitions of each split.

At first glance, the difference between 71.93% and 72.47% test accuracy may seem modest. However, the task is not a binary classification problem. At each decision step, the policy must choose among multiple candidate edges and a terminal action. More importantly, the learned policy outperforms both simple baselines by a large margin, which shows that its performance cannot be explained by naive low-weight selection or random guessing.

These next-action results support the claim that the proposed state representation and edge-wise policy are expressive enough to capture a substantial portion of the expert construction logic. In particular, the inclusion of normalized edge distances in both message passing and decoding is consistent with the weight-dependent nature of the target task and gives the policy direct access to information that is tightly related to the optimization objective.

C. Rollout Behavior

Next-action accuracy does not fully capture the structural quality of the learned policy. For this reason, an auxiliary rollout analysis was performed with the strongest highlighted checkpoint (Run 05) over the first 100 original graphs. This evaluation complements transition-level accuracy by testing whether locally predicted decisions accumulate into valid global graph constructions. In all 100 cases, the policy produced a valid tree and emitted the END action at the appropriate stage. Moreover, the predicted tree matched the expert MST cost in 52 of the 100 cases, and the average relative cost gap with respect to the expert MST was only 0.56%.

This result is important for two reasons. First, it shows that the policy does not merely memorize local edge patterns; it can sustain a coherent constructive trajectory until a complete

TABLE I

LAYER-WISE SUMMARY OF THE IMPLEMENTED POLICY ARCHITECTURE. HERE, B DENOTES BATCH SIZE, N THE PADDED NUMBER OF NODES, E THE NUMBER OF VALID UNDIRECTED EDGES IN ONE GRAPH, AND $E_{\max} = N(N - 1)/2$ THE GLOBAL PADDED EDGE BUDGET.

Stage	Operation	Input	Output	Role in the policy
Observed state	Dense matrix state	$(B, 3, N, N)$	$(B, 3, N, N)$	Three channels store the current tree, graph connectivity mask, and normalized edge distances.
Graph builder	Dense-to-sparse conversion	$(B, 3, N, N)$	$x, \text{edge_index}, \text{edge_attr}$	raw Extracts valid graph edges, builds sparse connectivity for PyG, and duplicates undirected edges in both directions for message passing.
Node projection	node_encoder: Linear($1, H$)	$(B \cdot N, 1)$	$(B \cdot N, H)$	Maps the constant scalar node feature to the hidden space used by the GNN backbone.
Edge projection	edge_encoder: Linear($2, H$)	$(2BE, 2)$	$(2BE, H)$	Projects raw edge attributes $[\text{tree}_{uv}, \text{dist}_{uv}]$ into hidden edge embeddings.
Message passing 1	GINEConv + LeakyReLU	node $(B \cdot N, H)$, edge $(2BE, H)$	$(B \cdot N, H)$	First edge-aware aggregation layer over the sparse graph.
Message passing 2	GINEConv + LeakyReLU	$(B \cdot N, H)$	$(B \cdot N, H)$	Refines node embeddings with a second round of edge-aware message passing.
Reshape	Batched node layout restoration	$(B \cdot N, H)$	(B, N, H)	Reorganizes flat node embeddings back into one tensor per graph for decoding.
Edge feature assembly	Concatenation of endpoint and raw edge features	node embeddings + raw edge features	$(B, E_{\max}, 2H + 2)$	Forms $\mathbf{z}_{uv} = [\mathbf{h}_u^{(2)}, \mathbf{h}_v^{(2)}, \mathbf{e}_{uv}]$ for every padded candidate edge slot.
Residual edge decoder	residual: Linear→LeakyReLU→Linear	$(B, E_{\max}, 2H + 2)$	$(B, E_{\max}, 2H + 2)$	Refines explicit edge representations before scalar scoring.
Edge scoring head	output: Linear($2H + 2, 1$)	$(B, E_{\max}, 2H + 2)$	$(B, E_{\max})$	Produces one logit for each padded edge position in the global action space.
Graph readout	Mean pooling over nodes	(B, N, H)	(B, H)	Compresses node embeddings into one graph-level representation for termination prediction.
END head	end_head: Linear→LeakyReLU→Linear	(B, H)	$(B, 1)$	Predicts the END logit that indicates when tree construction should stop.
Final action vector	Concatenation of edge logits and END logit	$(B, E_{\max})$ and $(B, 1)$	$(B, E_{\max} + 1)$	Produces the full padded action vector used during training and masked inference.

TABLE II
IMPLEMENTATION SUMMARY.

Item	Value
Python	3.11.4
PyTorch	2.13.0
PyTorch Geometric	2.8.0
NetworkX	3.1
NumPy	2.4.6
Hardware	Apple M1 Pro, 16 GB RAM
Accelerator backend	PyTorch MPS
Train/val/test split	80% / 10% / 10%
Split seed	42
Optimizer	Adam
Checkpoint criterion	Lowest validation loss
Grid search	12 configurations

TABLE III
DATASET CHARACTERISTICS.

Statistic	Value
Original graph episodes	100,000
Expanded transitions	1,283,100
Maximum number of nodes	20
Maximum number of edges	52
Maximum observed local action space	53
Network output dimension	191
Stored state precision	float16

TABLE IV
PERFORMANCE OF HIGHLIGHTED CHECKPOINTS AND SIMPLE BASELINES.

Model	Hidden	Val. Loss	Val. Acc.	Test Acc.
Random valid action	–	–	3.50%	3.50%
Greedy acyclic edge	–	–	31.72% [†]	32.24% [†]
Run 04	64	0.7187	71.90%	71.93%
Run 05	128	0.6953	72.37%	72.47%

[†]Estimated on the first 5,000 transitions of each split.

valid tree is formed. Second, even when the predicted tree is not exactly the expert MST, the cost difference is often small, which suggests that the policy has learned to prefer low-cost structural decisions.

A representative example was also inspected separately. In that rollout, the best checkpoint selected exactly $N - 1$ edges and terminated correctly, producing a complete spanning tree whose total cost was 0.63% above the expert MST cost. This qualitative behavior is consistent with the aggregate rollout analysis.

D. Limitations

The present results should be interpreted within the restricted scope of the benchmark. First, the learner is trained

to imitate one deterministic expert sequence, not to optimize tree cost directly. Since different locally plausible edges may exist at intermediate states, exact imitation is a stricter requirement than structural near-optimality. This partly explains why rollout quality can be strong even when next-action accuracy is not perfect. The ambiguity is structural: one graph may admit several equally optimal or

near-optimal construction sequences, while the dataset stores only one expert order. Therefore, some actions counted as misclassifications at the transition level can still lead to valid or even optimal final trees.

Second, the expert trajectory is not produced by an online constructive policy such as Prim or Kruskal acting step by step. Instead, the MST is first solved as a static structure and then converted into an ordered sequence by a deterministic depth-first traversal. This makes the supervision signal well defined and reproducible, but it also means that the learned policy imitates a particular serialization of the solution rather than a unique notion of optimal constructive behavior.

Third, the empirical evaluation is intentionally limited. Training and test graphs are generated by the same synthetic process, all experiments remain within the 8-to-20-node regime, and the rollout analysis is reported on the first 100 original graphs as a supplementary structural check rather than as a large-scale statistical study. For the same reason, the reported baselines are simple reference procedures, not a comprehensive benchmark against stronger constructive heuristics or alternative neural architectures. Therefore, the current evidence supports the claim of a controlled supervised baseline, but not a broader claim of structural generalization across graph families or of competitiveness against mature optimization methods.

Fourth, the implementation adopts a fixed padded global action space with 191 logits, of which only a subset is valid for each graph instance. This choice is convenient for batching and matches the current implementation, but it ties the policy to the preset node budget and does not by itself address scalability to substantially larger graphs.

These limitations do not invalidate the benchmark; rather, they clarify its role. The central contribution of the present work is to show that a GNN-based behavior-cloning pipeline can learn non-trivial edge-selection behavior in a controlled MST setting. Stronger claims, such as broad structural generalization, direct topology-cost optimization, or scalable deployment beyond the current regime, remain outside the evidence produced here.

IV. CONCLUSION

This work investigated whether deep learning can be used to imitate graph-construction trajectories in the specific benchmark setting of Minimum Spanning Trees. To answer this question, a complete behavior-cloning pipeline was designed, implemented, and evaluated, including weighted graph generation, deterministic MST labeling, transition expansion, graph neural policy learning, and highlighted-checkpoint evaluation.

Within this controlled benchmark, the objective of the work was achieved. The proposed system learned a non-trivial policy that maps partial graph states to edge-selection actions and can produce valid spanning-tree rollouts with low average cost deviation from the expert MST. These results support the narrower claim advanced in this paper: MST construction can be reformulated as a supervised sequential decision problem, and a GNN-based policy can learn a

substantial portion of the associated deterministic expert behavior while preserving weighted structural information.

The distinction between central and additional contributions is important for interpreting the scope of the paper. The central contribution is the supervised benchmark itself: the formulation, representation, implementation, and controlled empirical evaluation of an edge-by-edge MST imitation pipeline. Additional contributions, which are motivated by the present study but not demonstrated here, should be understood as future extensions rather than as completed outcomes.

Accordingly, several directions remain open for future work. First, the supervised policy can be used as initialization for reinforcement learning, allowing direct optimization of global topology cost instead of exact imitation of one expert order [?], [?], [?]. Second, the trajectory definition itself can be revisited, for example through alternative constructive experts or supervision targets that better reflect equivalence between multiple valid edge orders. Third, stronger evaluation protocols should be added, including broader rollout studies, multiple random seeds, stronger baselines, and experiments on graph families that go beyond the current synthetic IID regime. Fourth, larger, denser, or more irregular graphs should be explored to assess scalability and robustness under more demanding structural conditions [?], [?], [?]. Finally, richer policy architectures and multi-objective training settings may be investigated once the benchmark is extended beyond the present controlled scope [?], [?].